Cours : Structures de Données en C#

Dans ce cours, nous allons explorer les structures de données fondamentales proposées par C#. Une bonne maîtrise de ces structures est essentielle pour écrire des programmes efficaces et optimisés.

Par Robert DIASSÉ

---

1. Introduction aux Structures de Données

Les **structures de données** permettent de stocker, organiser et manipuler des ensembles de données en mémoire. Elles sont cruciales pour optimiser les performances d'un programme en fonction des opérations à effectuer.

Catégories de Structures de Données

- **Structures linéaires :** Tableau (`Array`), Liste (`List`), Pile (`Stack`), File (`Queue`)
- **Structures non linéaires :** Dictionnaire (`Dictionary`), Ensemble (`HashSet`)

---

2. Les Tableaux (`Array`)

Définition :

Un tableau est une structure de données fixe qui stocke plusieurs éléments de même type dans une séquence continue en mémoire.

Déclaration et Initialisation

```
int[] nombres = new int[5];  // Déclare un tableau de 5 entiers
nombres[0] = 10;  // Affectation d'une valeur au premier élément
```

Initialisation Simplifiée

```
int[] nombres = { 10, 20, 30, 40, 50 };  // Déclaration et initialisation directe
```

Accès et Modification

```
Console.WriteLine($"Première valeur : {nombres[0]}"); // Affiche 10
nombres[0] = 100;
Console.WriteLine($"Nouvelle première valeur : {nombres[0]}"); // Affiche 100
```

Itération sur un Tableau

```
foreach (int nombre in nombres)
{
    Console.WriteLine(nombre);
}
```

---

3. Les Listes (`List<T>`)

Définition :

Une liste est une structure dynamique qui peut stocker un nombre variable d'éléments de même type.

Déclaration et Initialisation

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main(string[] args)
    {
        List<string> fruits = new List<string>() { "Pomme", "Banane", "Orange" };
        fruits.Add("Mangue");  // Ajout d'un nouvel élément
        Console.WriteLine($"Nombre de fruits : {fruits.Count}");
    }
}
```

Principales Méthodes

Méthode	Description
`Add()`	Ajoute un élément à la fin
`Remove()`	Supprime la première occurrence d'un élément
`Insert()`	Insère un élément à une position donnée
`Clear()`	Supprime tous les éléments

---

4. Les Dictionnaires (`Dictionary<K, V>`)

Définition :

Un dictionnaire est une collection qui associe une clé (`Key`) à une valeur (`Value`). Chaque clé doit être unique.

Déclaration et Utilisation

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main(string[] args)
    {
        Dictionary<string, int> ages = new Dictionary<string, int>
        {
            { "Alice", 25 },
            { "Bob", 30 },
            { "Charlie", 35 }
        };

        Console.WriteLine($"Âge de Bob : {ages["Bob"]} ans");

        // Vérification d'une clé
        if (ages.ContainsKey("Alice"))
        {
            Console.WriteLine("Alice existe dans le dictionnaire");
        }
    }
}
```

---

5. Les Piles (`Stack<T>`)

Définition :

Une pile est une structure de données LIFO (*Last In, First Out*), où le dernier élément ajouté est le premier retiré.

Exemple d'Utilisation

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main(string[] args)
    {
        Stack<string> historique = new Stack<string>();

        historique.Push("Page 1");
```

```
        historique.Push("Page 2");
        historique.Push("Page 3");

        Console.WriteLine($"Dernière page consultée : {historique.Pop()}"); //
Affiche Page 3
    }
}
```

---

6. Les Files (`Queue<T>`)

Définition :

Une file est une structure de données FIFO (*First In, First Out*), où le premier élément ajouté est le premier retiré.

Exemple d'Utilisation

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main(string[] args)
    {
        Queue<string> filesAttente = new Queue<string>();

        filesAttente.Enqueue("Client 1");
        filesAttente.Enqueue("Client 2");
        filesAttente.Enqueue("Client 3");

        Console.WriteLine($"Client servi : {filesAttente.Dequeue()}"); // Affiche
Client 1
    }
}
```

---

7. Exemples Pratiques

Exemple 1 : Liste des Nombres Pairs

Écrire un programme qui stocke les nombres pairs de 1 à 10 dans une liste et les affiche.

Solution :

```
using System;
using System.Collections.Generic;
```

```
class Program
{
    static void Main(string[] args)
    {
        List<int> nombresPairs = new List<int>();

        for (int i = 1; i <= 10; i++)
        {
            if (i % 2 == 0)
            {
                nombresPairs.Add(i);
            }
        }

        Console.WriteLine("Nombres pairs de 1 à 10 :");
        foreach (int nombre in nombresPairs)
        {
            Console.WriteLine(nombre);
        }
    }
}
```

Exemple 2 : Gestion d'un Dictionnaire

Créer un programme qui stocke les noms et âges d'un groupe de personnes dans un dictionnaire et affiche les informations.

Solution :

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main(string[] args)
    {
        Dictionary<string, int> personnes = new Dictionary<string, int>
        {
            { "Alice", 25 },
            { "Bob", 30 },
            { "Charlie", 35 }
        };

        foreach (var personne in personnes)
        {
            Console.WriteLine($"Nom : {personne.Key}, Âge : {personne.Value}
ans");
        }
    }
}
```

---

8. Exercices à Faire

1. **Tableau Statique :**
   Écrire un programme qui stocke 5 notes saisies par l'utilisateur dans un tableau et calcule la moyenne.

2. **Liste Dynamique :**
   Demander à l'utilisateur d'entrer des prénoms jusqu'à ce qu'il saisisse "fin". Afficher ensuite la liste complète des prénoms.

3. **Pile :**
   Simuler une gestion de navigation de pages web en utilisant une pile. Afficher l'historique des pages consultées.

4. **File d'Attente :**
   Écrire un programme qui simule une file d'attente dans une banque avec trois clients. Servir les clients un par un.

---

9. Concepts Clés à Retenir

- Les **Tableaux** permettent de stocker des éléments de taille fixe.
- Les **Listes** sont dynamiques et offrent une grande flexibilité.
- Les **Dictionnaires** permettent une recherche rapide via des clés uniques.
- Les **Piles** et **Files** permettent de gérer les données selon des logiques spécifiques (LIFO et FIFO).